

SOFTWARE DESIGN SPECIFICATION (SDS)

Project Title:

Ordinis

A Dynamic Multi-Tenant Organization Workflow & Chain-of-Command Management System

SL	Student ID	Student Name	Role
1	23201008	Fawjiya Fahima Eifa	System Designer / Software Architect
2	23201023	Md. Nazmuz Saif	Software Developer (Programmer)
3	23201029	Sadia Jahan Disha	System Analyst / Business Analyst
4	23201050	Md. Tahsin Rahman Khan	Tester / Quality Assurance (QA) Engineer
5	23201055	Montasim Kabir Mahin	Project Manager

Document Version	Date of Submission
1.0	10.09.2026

Table of Contents

Document History

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Intended Audience

1.4 References

2. Design Overview

2.1 Design Goals

2.2 Design Principles

2.3 System Context

3. System Architecture

3.1 Architectural Overview

3.2 Architectural Diagram

3.3 Technology Stack

3.4 Deployment Architecture

4. Data Design

4.1 Data Model Overview

4.2 Entity Relationship Diagram (ERD)

4.3 Data Dictionary

4.4 Data Storage & Retention

5. Component Design

5.1 Component Overview

5.2 Detailed Component Design

Component Name:

6. Interface Design

6.1 User Interface Design

6.2 External Interfaces

6.3 API Design

6.4 Hardware Interfaces (if applicable)

6.5 Communication Interfaces

7. Security Design

7.1 Authentication & Authorization

7.2 Data Security

7.3 Security Threat Analysis

8. Performance Design

8.1 Performance Requirements

8.2 Scalability Strategy

8.3 Capacity Planning

9. Error Handling & Logging
9.1 Error Handling Strategy
9.2 Logging Strategy
10. Deployment Design
10.1 Deployment Strategy
10.2 Environment Configuration
10.3 Rollback Strategy
11. Design Constraints
12. Assumptions & Dependencies
13. Traceability Matrix
14. Appendices
Appendix A – Glossary
Appendix B – Diagrams
Appendix C – Supporting Documents

Version	Date	Authors	Change Log	Approved By
V1.0		Montasim Kabir Mahin Fawjiya Fahima Eifa Sadia Jahan Disha Md.Tahsin Rahman Khan Md. Nazmuz Saif	Initial SDS based on the approved Ordinis requirements and design scope	

1. Introduction

1.1 Purpose

This Software Design Specification (SDS) provides a detailed technical description of the architecture, system components, data models, and interface designs for **Ordinis**—a dynamic, multi-tenant Workflow and Chain-of-Command Configuration Platform. The purpose of this document is to serve as the single source of truth for translating functional requirements into system implementation.

This document covers:

- High-level software architecture and physical deployment design.
- Data models, database schemas, and data dictionaries.
- Module component logic, interaction sequences, and service-layer patterns.
- External interfaces, REST API design, and UI standards.
- System security, performance, error handling, and auditability mechanisms.

1.2 Scope

Ordinis is a web-based, multi-tenant SaaS "Organization Operating System" (OrgOS). The platform enables client organizations to dynamically define and manage their operational hierarchies, role-based access levels, and approval workflows without requiring code modifications or legacy system dependencies.

System Goals & Objectives:

Dynamic Hierarchy Configuration: Enable client companies to construct arbitrary-depth organizational structures (Departments, Designations, and reporting relationships).

Decoupled Access Control: Enforce Role-Based Access Control (RBAC) where operational permissions are tied to explicit role bundles rather than rigid job designations.

Configurable Workflow Engine: Provide a data-driven engine to route multi-step approvals (e.g., leave, expense) and track delegable authority.

Core Operational Modules: Automate task/project tracking, attendance monitoring, leave requests, and finance/payroll processing.

Multi-Tenant Isolation & Auditability: Guarantee multi-tenant data isolation at the storage layer while maintaining an append-only audit trail for administrative transparency.

1.3 Intended Audience

This document is structured for the following technical and operational stakeholders:

Software Developers & Engineers: To guide code implementation of backend APIs, business services, and frontend UI components.

System Architects: To review architectural patterns, service separation, and database performance strategies.

Quality Assurance (QA) Engineers: To design unit, integration, security, and regression test suites based on defined inputs, outputs, and constraints.

Course Evaluators & Project Reviewers: To verify system structure, design completeness, and alignment with academic/software engineering standards.

DevOps & System Administrators: To configure environment settings, deployment pipelines, and database retention strategies.

1.4 References

The following documents served as foundational inputs, standards, and specifications in the design of the **Ordinis** system:

IEEE Std 1016-2009: IEEE Standard for Information Technology—Systems Design—Software Design Descriptions.

IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.

Software Requirements Specification (SRS): Ordinis — A Dynamic, Multi-Tenant Workflow & Chain-of-Command Configuration Platform (V1.0).

Business Requirements Document (BRD): Ordinis Project Proposal and Initial Business Vision Specifications.

Database & Data Design Specifications: Ordinis Entity-Relationship (ER) Diagram and Master Data Schema.

Security & Error Handling Guidelines: Ordinis System Architecture, Tenant Isolation, and Error Handling Standards.

Frontend UI/UX Standards: Ordinis Design System and Dashboard Design Guidelines.

2. Design Overview

2.1 Design Goals

Scalability: The architecture must support multi-tenant horizontal growth, allowing new client companies (tenants) and additional employees to be onboarded without degrading performance or requiring schema changes.

Maintainability: Business logic, data models, and presentation interfaces must be cleanly isolated so that system features, approval workflows, or core business rules can be updated or extended with minimal code friction.

Security: Strict multi-tenant data isolation must be enforced at every layer (ensuring no cross-tenant data leakage), combined with robust JWT authentication, fine-grained permission checks, and encrypted communication.

Performance: API endpoints, list queries, and authorization checks must be optimized with pagination and light data execution to maintain low response times under active operational loads.

Availability: System operations must remain accessible for daily organizational tasks (such as check-ins, task reviews, and approvals) with continuous uptime and transparent error-handling strategies.

2.2 Design Principles

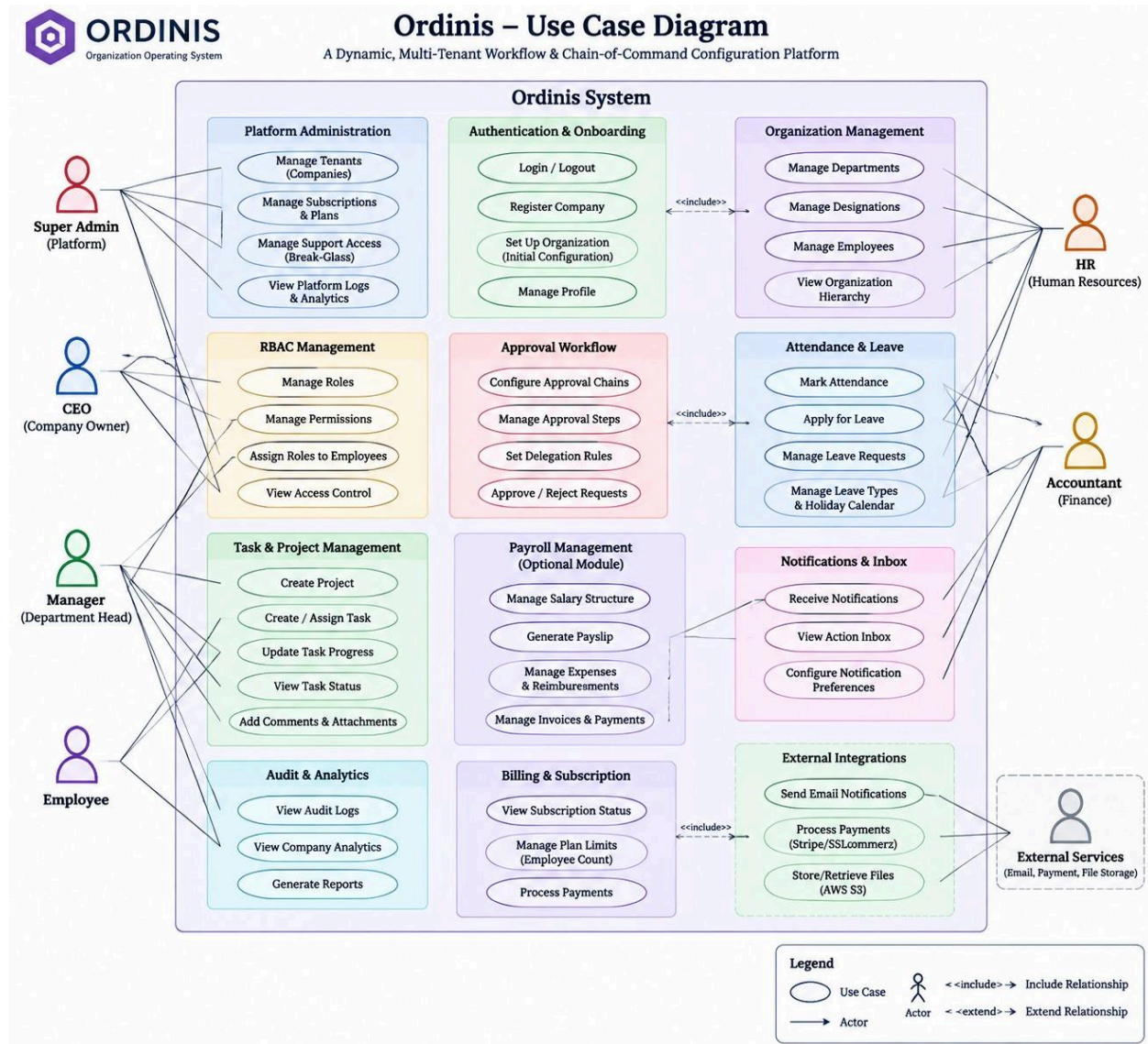
Separation of Concerns: Distinct application layers—Presentation (React SPA), API Routing (DRF), Service Logic ([services.py](#)), and Data Access (Django ORM)—must operate independently without overlapping responsibilities.

Modularity: System functionalities are divided into self-contained, domain-specific modules (Auth, Hierarchy, Approvals, Tasks, Attendance, Payroll, Audit) that can be built, tested, and maintained independently.

Reusability: Common workflow patterns—such as permission verification, audit logging, data-driven approval steps, and response formatting—are centralized into reusable utility services, middlewares, and mixins.

High Cohesion / Low Coupling: Related functionalities are tightly grouped within their respective domain modules while inter-module interactions rely strictly on well-defined service interfaces and permissions.

2.3 System Context



3. System Architecture

3.1 Architectural Overview

Ordinis uses a **Layered Architecture (N-Tier)** paired with a **Client-Server Single-Page Application (SPA) Pattern**. This architectural pattern enforces strict separation of concerns, ensuring high cohesion and low coupling across the application layers.

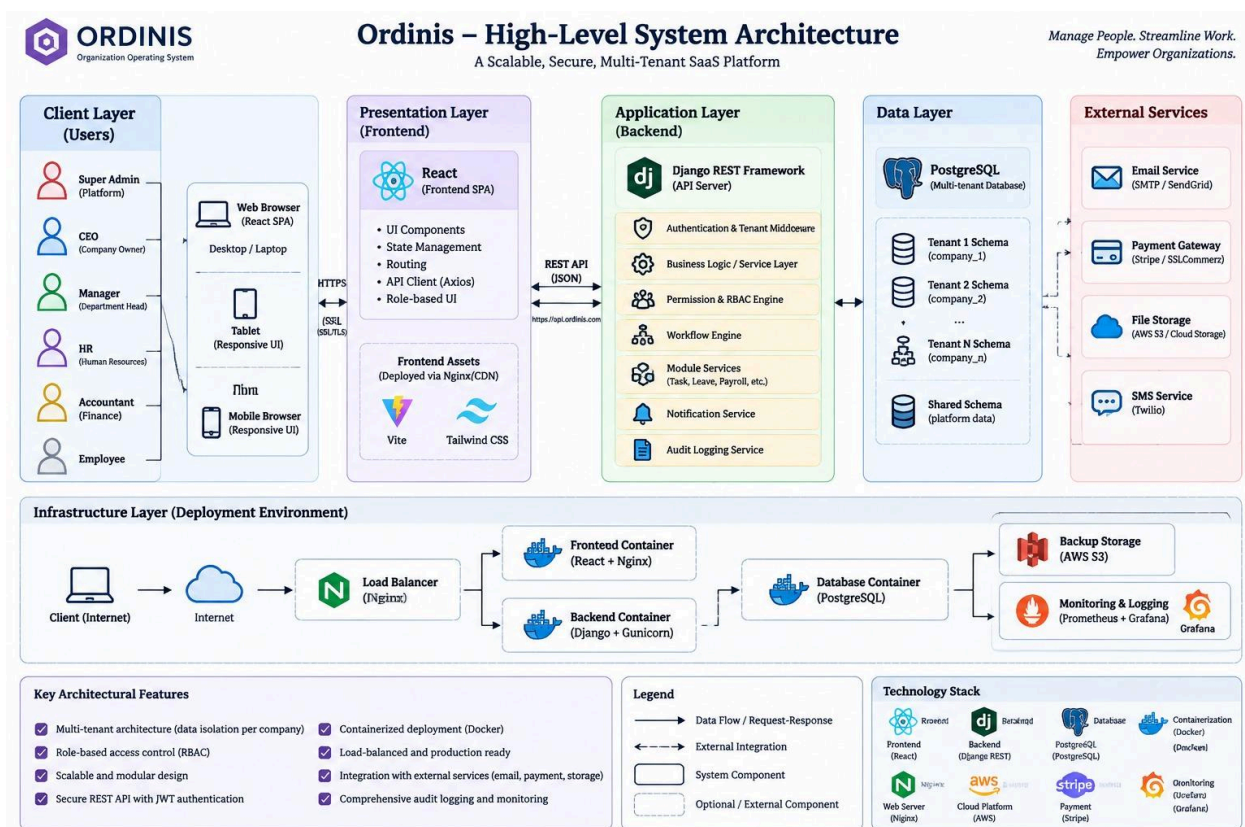
The core architectural layers consist of:

- 1. Presentation Layer:** A responsive React Single-Page Application (SPA) built with Vite and styled using Tailwind CSS. It handles user interactions, renders role-based contextual

UI layouts, and communicates asynchronously with the backend via versioned REST API endpoints.

2. **API & Controller Layer:** Implemented using Django REST Framework (DRF). It manages HTTP request routing, JWT token authentication, request payload validation, and standard JSON envelope formatting.
3. **Service & Core Engine Layer (*services.py*):** Encapsulates all domain-specific business rules, dynamic hierarchy resolutions, approval workflow processing, tenant isolation enforcement, and audit logging. Keeping business logic inside services ensures thin HTTP controllers and prevents business logic duplication.
4. **Data Access Layer (ORM):** Utilizes Django's Object-Relational Mapping (ORM) to abstract database operations. Every database query is programmatically scoped by *company_id* to guarantee multi-tenant data isolation.
5. **Persistence & Storage Layer:** A relational database system storing operational entity data alongside append-only audit trails.

3.2 Architectural Diagram



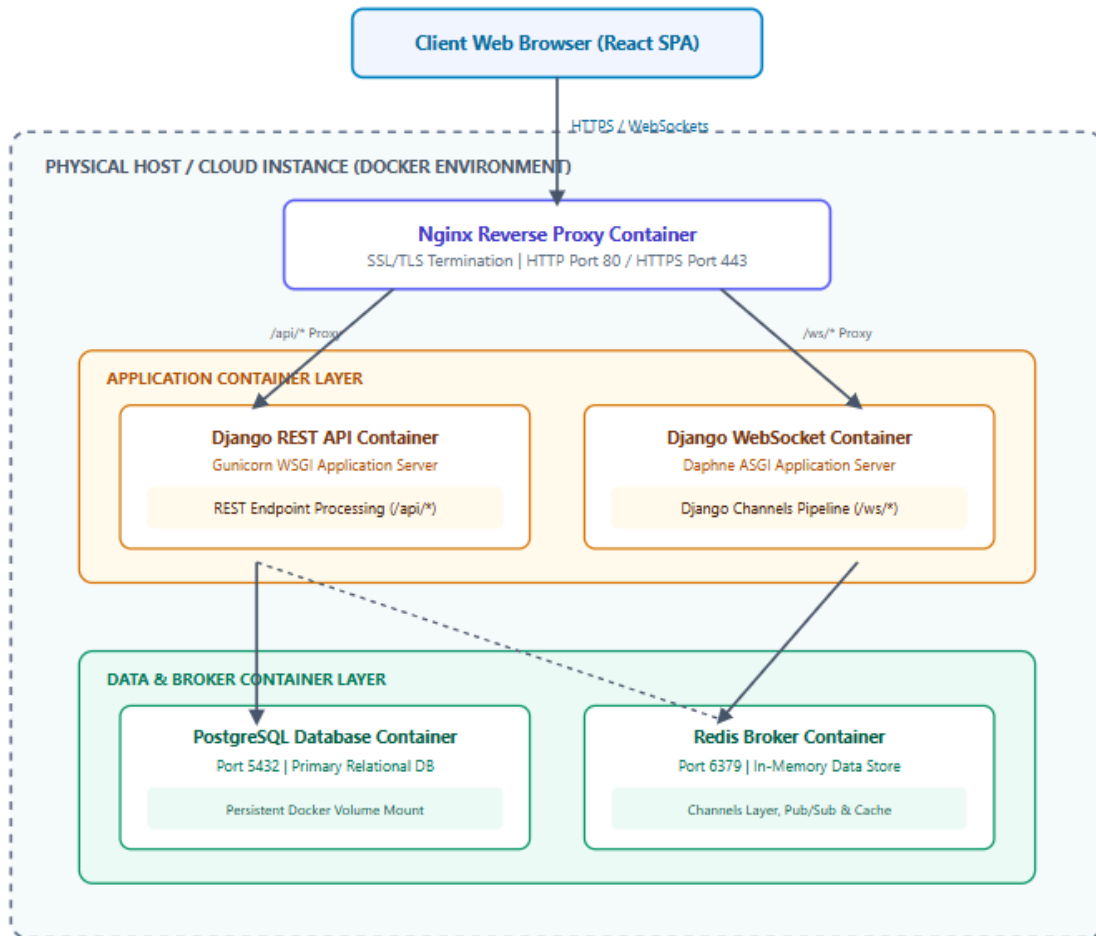
3.3 Technology Stack

Layer	Technology	Version
Frontend	React (Vite Build Tool), Tailwind CSS, Axios / Native Fetch API	React 18.x ,Tailwind v3.x ,ES2022+
Backend	Python ,Django ,Django REST Framework (DRF)	Python 3.10+ ,Django 4.2+ LTS ,DRF 3.14+
Database	SQLite (Development / Local Testing), PostgreSQL (Production Target)	SQLite 3.x, PostgreSQL 15+
Infrastructure	Gunicorn (WSGI Server) ,Nginx (Reverse Proxy & Static File Host) ,Docker / Linux Virtual Private Server (VPS)	WSGI Standard ,Nginx 1.24+ , Docker Engine 24+

3.4 Deployment Architecture

Ordinis is designed for containerized deployment across cloud infrastructure (AWS/GCP/Azure) or managed on-premise Docker environments. The platform utilizes a microservice-style container orchestration approach using Docker and Docker Compose (or Kubernetes for enterprise scaling) to ensure complete environment isolation, reproducibility, and high availability .

Ordinis - Deployment & Infrastructure Architecture



Deployment Components Breakdown:

Host & Infrastructure Layer: Deployed on standard cloud Virtual Machines (AWS EC2, DigitalOcean, Azure VM) or local Linux servers running Docker Engine and Docker Compose.

Web Gateway Container (Ngix): Acts as the entry point managing incoming HTTPS traffic on Port 443. Handles SSL/TLS certificate termination, static asset delivery, and routes REST traffic to WSGI workers and WebSocket traffic to ASGI workers.

REST Service Container (Gunicorn + Django): Runs multiple Gunicorn worker processes handling stateless HTTP REST API calls and core business controller executions.

Real-time Service Container (Daphne + Channels): Runs Daphne ASGI workers to sustain persistent, low-latency WebSocket connections for live Action Inbox notifications.

Data Storage Containers (PostgreSQL & Redis):

- PostgreSQL manages persistent data mounted to isolated host volumes for data backup and safety.
- Redis acts as an in-memory message broker for Django Channels communication and application-wide caching.

4. Data Design

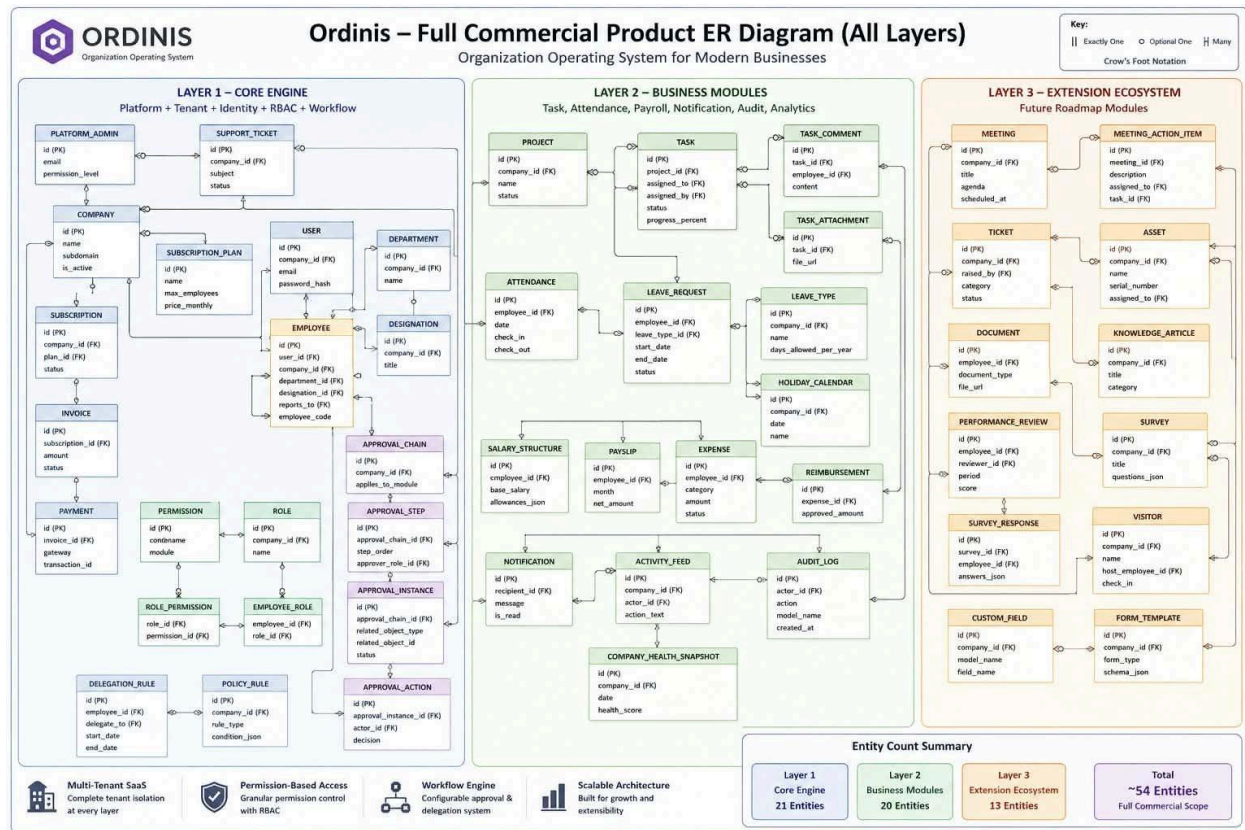
4.1 Data Model Overview

Ordinis employs a **Relational Database Model (RDBMS)**. A relational approach is required due to the platform's high structural complexity, strict data integrity constraints, transactional consistency requirements (ACID compliance for payroll and leave operations), and complex relational entities.

Key characteristics of the data model include:

- **Multi-Tenant Partitioning:** Every domain table contains a foreign key reference ([company_id](#)). Data isolation is logically enforced across all queries.
- **Dynamic Hierarchy via Self-Referencing Foreign Keys:** The employee hierarchy tree is represented using a self-referencing foreign key ([reports_to](#)) pointing to another [Employee](#) record within the same tenant.
- **Decoupled Access Control:** Roles and permissions are represented using a many-to-many relationship mapping ([Employee](#) – [Role](#) – [Permission](#)), decoupled from job titles ([Designation](#)).
- **Append-Only Logging:** Audit trails ([AuditLog](#)) and support access logs ([ImpersonationLog](#)) are structured as append-only records with no update or deletion endpoints.

4.2 Entity Relationship Diagram (ERD) :



4.3 Data Dictionary

The Data Dictionary defines the main data entities used in the Ordinis system. It describes the important fields, their data types, and their purpose.

Entity	Field	Data Type	Description
Company (Tenant)	company_id	Integer	Unique identifier of the company.
Company (Tenant)	name	String	Name of the company.
Department	department_id	Integer	Unique identifier of a department.

Department	name	String	Name of the department.
Designation	designation_id	Integer	Unique identifier of a designation.
Designation	title	String	Job title of an employee.
Employee	employee_id	Integer	Unique identifier of an employee.
Employee	user_id	Integer	Links the employee with a system user account.
Employee	reports_to	Integer / NULL	References the employee's manager in the organizational hierarchy.
Employee	department_id	Integer	Identifies the employee's department.
Role	role_id	Integer	Unique identifier of a role.
Role	name	String	Name of the role.
Permission	permission_id	Integer	Unique identifier of a permission.
Permission	codename	String	Unique permission code checked by the backend.
EmployeeRole	employee_id	Integer	Identifies the employee assigned to a role.
EmployeeRole	role_id	Integer	Identifies the assigned role.
Approval Chain	chain_id	Integer	Unique identifier of an approval chain.
Approval Step	step_id	Integer	Unique identifier of an approval step.
Approval Instance	instance_id	Integer	Identifies a running approval process.
Approval Action	action_id	Integer	Stores an action taken during approval.

Delegation Rule	delegation_id	Integer	Identifies an approval delegation rule.
Project	project_id	Integer	Unique identifier of a project.
Task	task_id	Integer	Unique identifier of a task.
Task	status	String	Current state of the task.
Task Progress Log	progress_id	Integer	Records daily task progress.
Attendance	attendance_id	Integer	Unique attendance record identifier.
Attendance	date	Date	Date of the attendance record.
Leave Type	leave_type_id	Integer	Identifies a type of leave.
Leave Request	leave_id	Integer	Unique leave request identifier.
Leave Request	status	String	Current approval status of the leave request.
Salary Structure	salary_id	Integer	Identifies an employee salary structure.
Payslip	payslip_id	Integer	Unique identifier of a generated payslip.
Expense	expense_id	Integer	Unique expense record identifier.
Reimbursement	reimbursement_id	Integer	Identifies a reimbursement request.
Notification	notification_id	Integer	Unique notification identifier.
Audit Log	audit_id	Integer	Unique identifier of an audit record.
Audit Log	action	String	Records the action performed by a user or system process.

Subscription	subscription_id	Integer	Identifies the company's subscription.
Invoice	invoice_id	Integer	Identifies a billing invoice.
Support Access Request	request_id	Integer	Identifies a break-glass support access request.
Impersonation Log	log_id	Integer	Records approved support access activity.

The database uses relationships between these entities to maintain organizational structure, permissions, workflow, attendance, leave, payroll, and audit information.

4.4 Data Storage & Retention

Ordinis uses a relational database for storing structured organizational and workflow data. The Django ORM is used to access and manage the database.

4.4.1 Data Storage

- The development environment uses **SQLite** for simple local development and testing.
- The production environment targets **PostgreSQL** for reliable multi-tenant data storage.
- Each major business record is associated with a company/tenant.
- Tenant-specific queries are always filtered using the appropriate **company_id**.
- Relational constraints and foreign keys are used to maintain data consistency.
- Transactions are used for important operations such as leave, attendance, and payroll updates.
- Sensitive authentication data is stored using secure one-way password hashing.
- Access tokens are short-lived, while refresh tokens are rotated according to the authentication design.
- Audit records are stored separately as append-only records to preserve accountability.

4.4.2 Data Retention

Ordinis retains data according to its business purpose and operational requirements.

Data Type	Storage	Retention Approach
User and Employee Data	Relational Database	Retained while the employee/company account is active and according to business requirements.
Organization Data	Relational Database	Retained while the company account exists.
Attendance Records	Relational Database	Retained for operational and reporting purposes according to company policy.
Leave Records	Relational Database	Retained for leave history and reporting.
Payroll Records	Relational Database	Retained for salary and financial history according to business requirements.
Task and Project Data	Relational Database	Retained for project tracking and historical reference.
Approval Records	Relational Database	Retained to maintain workflow history and accountability.
Notifications	Relational Database	Retained for a reasonable operational period and can be cleaned up when no longer required.
Audit Logs	Append-only Database Records	Retained for security, accountability, and compliance purposes.

Subscription and Billing Data	Relational Database	Retained for subscription and billing history.
Temporary Support Access Logs	Relational Database	Retained to provide an audit trail of break-glass access.

4.4.3 Backup and Recovery

- Production database backups should be performed regularly.
- Backups should be protected from unauthorized access.
- Recovery procedures should be tested periodically.
- Data restoration should maintain tenant separation and database consistency.
- Critical transactional operations should be designed so that incomplete operations do not leave inconsistent data.

4.4.4 Data Deletion

When data is no longer required, it may be removed according to the company's retention policy and system rules. However, security and audit records should not be casually deleted because they are required for accountability. Any supported deletion process should respect tenant boundaries and maintain database.

5. Component Design

5.1 Component Overview

Component Name	Description	Responsibility
Authentication & Tenant Onboarding	Identity and multi-tenant registration service	Manages secure authentication (JWT), password hashing, user sessions, and initial company onboarding.

Dynamic Hierarchy & RBAC	Organization structure and permission engine	Handles manager reporting trees (reports_to) and enforces granular, role-based access control (RBAC).
Approval Workflow	Dynamic multi-step request processing engine	Routes requests through configurable approval steps, tracks decision actions, and handles delegation rules.
Task & Project Management	Task lifecycle and tracking system	Enforces task state transitions, manages task assignments, and captures daily progress percentage logs.
Attendance & Leave Management	Time tracking and leave balance engine	Manages daily check-ins, calculates leave balance availability, and routes leave applications for approval.
Payroll & Expense Management	Compensation and reimbursement processor	Computes monthly payslips, handles salary structures, and executes expense reimbursement flows.
Audit Logging & Security	Data modification and support access logger	Records immutable activity logs across data models and manages temporary "Break-Glass" support impersonation.

5.1(Component Overview)

1. Authentication & Tenant Onboarding Component

Purpose: Provides secure user authentication, handles token lifecycles, and manages multi-tenant organization onboarding.

Inputs: User login credentials (email, password), tenant metadata, CEO account registration payload.

Outputs: Signed JWT access/refresh token pair, basic profile data, status codes.

Dependencies: Data Access Layer (Django ORM), django-rest-framework-simplejwt.

Internal Logic:

1. Validate incoming user registration or login payloads.
2. Perform atomic database creation of Company, CompanySettings, initial Designation, and default CEO Employee record.
3. Validate password hashes on authentication requests using PBKDF2 algorithm.
4. Verify company active status (`is_active == True`); block login if tenant is suspended.
5. Issue JWT token containing encoded `user_id` and `company_id` claims.

Sequence Diagram:

1. **Client** → **API Controller**: Submits login request POST `/api/v1/auth/login/` containing user email and password.
2. **API Controller** → **Auth Service**: Forwards credentials for verification.
3. **Auth Service** → **Database (ORM)**: Queries user record matching the provided email address.
4. **Database (ORM)** → **Auth Service**: Returns user database record along with password hash and tenant linkage.
5. **Auth Service**: Performs PBKDF2 password verification and verifies that the tenant status is active (`is_active == True`).
6. **Auth Service** → **API Controller**: Generates and returns signed JWT access/refresh token pair.
7. **API Controller** → **Client**: Returns 200 OK response payload with access token, refresh token, and user session context.

2. Dynamic Hierarchy & RBAC Component

Purpose: Manages dynamic organizational reporting chains (`reports_to`), departments, job titles, and permission checks decoupled from job titles.

Inputs: Department and designation records, employee manager mappings (`reports_to_id`), role permissions.

Outputs: Hierarchical structures, permission authorization statuses, scoped data lists.

Dependencies: Authentication Component, Data Access Layer (Django ORM).

Internal Logic:

1. Store reporting trees using self-referencing foreign key relationships (Employee.reports_to).
2. Resolve access rights by inspecting active Role permission codenames assigned to an employee.
3. Dynamically restrict operational data queries based on employee hierarchy position or explicit permission rules.

Sequence Diagram:

1. **Client** → **API Controller**: Submits operational request with HTTP bearer authorization token.
2. **API Controller** → **RBAC Service**: Resolves user_id and company_id from token and requests permission check (perm_codename).
3. **RBAC Service** → **Database (ORM)**: Queries assigned permission bundles and active roles (Employee → Role → Permission).
4. **Database (ORM)** → **RBAC Service**: Returns assigned role mappings and permission list for the active user.
5. **RBAC Service**: Evaluates if requested perm_codename exists within the user's granted permissions.
6. **RBAC Service** → **API Controller**: Returns authorization outcome status (Granted or Denied).
7. **API Controller** → **Client**: Responds with 200 OK and requested dataset if authorized, or 403 Forbidden if denied.

3. Approval Workflow Component

Purpose: Executes dynamic multi-step approval routing and manages delegable approval authority.

Inputs: Entity request instances (leave, expense, task), approver decisions (Approve/Reject), delegation rules.

Outputs: Updated approval instance states (Pending, Approved, Rejected), logged audit approval actions.

Dependencies: RBAC Component, Notification Component, Data Access Layer.

Internal Logic:

1. Instantiate an ApprovalInstance on request creation and link initial target ApprovalStep.

2. Evaluate active DelegationRule entries if designated primary approver is absent.
3. Record every decision entry into an append-only ApprovalAction history table.
4. Advance step index upon step approval or transition instance to Rejected on denial.

Sequence Diagram:

1. **Approver → API Controller:** Submits decision action (Approve or Reject) with comments via POST /api/v1/approvals/{id}/action/.
2. **API Controller → Approval Service:** Validates approver identity and forwards decision parameters.
3. **Approval Service → Database (ORM):** Checks active ApprovalStep configuration and active delegation rules (DelegationRule).
4. **Database (ORM) → Approval Service:** Confirms valid approver authority for the target step.
5. **Approval Service → Database (ORM):** Inserts immutable record into ApprovalAction table.
6. **Approval Service → Database (ORM):** Increments workflow step index if approved, or marks overall request state as Rejected if denied.
7. **Approval Service → API Controller:** Confirms request state transition complete.
8. **API Controller → Approver:** Returns 200 OK success message with updated workflow state payload.

4. Task & Project Management Component

Purpose: Controls task tracking, state machine status validations, and daily progress log creation.

Inputs: Task parameters (title, deadline, assignee), status transitions, progress update records.

Outputs: Task state changes, progress percentage logs, historical activity lists.

Dependencies: Hierarchy Component, Notification Component, Data Access Layer.

Internal Logic:

1. Enforce strict state transition rules (Not Started → In Progress → Submitted → Completed).
2. Validate update permissions: permit write updates to assigned users and review actions to managers.
3. Store daily execution logs in TaskProgressLog with updated percentage indicators.

Sequence Diagram:

1. **Assignee → API Controller:** Sends progress log entry containing completion percentage and status updates.
2. **API Controller → Task Service:** Validates input payload and checks assignee ownership.
3. **Task Service:** Verifies status transition against state machine rules (e.g., verifying transition from In Progress to Submitted).
4. **Task Service → Database (ORM):** Saves record to TaskProgressLog and updates overall Task entity state.
5. **Database (ORM) → Task Service:** Confirms successful database persistence.
6. **Task Service → API Controller:** Returns newly updated task record and activity log.
7. **API Controller → Assignee:** Responds with 200 OK success envelope and updated task payload.

5. Attendance & Leave Management Component

Purpose: Handles daily check-in/out timestamps, leave balance validations, and leave application routing.

Inputs: Daily check-in triggers, leave applications (leave_type, date range, reason).

Outputs: Timestamped attendance entries, leave balance adjustments, created request entities.

Dependencies: Approval Workflow Component, Hierarchy Component, Data Access Layer.

Internal Logic:

1. Restrict employee check-ins to a single active daily record based on company time zone settings.
2. Compute requested leave days against remaining LeaveType balance; block applications exceeding remaining balance.
3. Dispatch valid leave applications to the Approval Workflow Component and adjust balance upon final approval.

Sequence Diagram:

1. **Employee → API Controller:** Submits leave application payload specifying leave_type_id, start_date, end_date, and reason.
2. **API Controller → Leave Service:** Forwards payload for balance validation and processing.
3. **Leave Service → Database (ORM):** Queries available leave balance for the requesting employee and target leave type.
4. **Database (ORM) → Leave Service:** Returns current remaining balance figures.

5. **Leave Service:** Verifies that requested duration does not exceed current remaining balance.
6. **Leave Service → Database (ORM):** Inserts LeaveRequest record and triggers initial workflow step in Approval Workflow Component.
7. **Leave Service → API Controller:** Returns confirmation of created leave request.
8. **API Controller → Employee:** Responds with 201 Created status code and pending request details.

6. Payroll & Expense Management Component

Purpose: Calculates employee salary structures, computes monthly payslips, and handles expense reimbursement processing.

Inputs: Base salary parameters, pay period parameters (month, year), expense reimbursement requests.

Outputs: Payslip records (gross salary, deductions, net salary), reimbursement payment records.

Dependencies: Attendance Component, RBAC Component, Data Access Layer.

Internal Logic:

1. Guarantee idempotent payslip processing by preventing duplicate records for the same employee and period.
2. Restrict all financial endpoints and records strictly to users assigned explicit finance permissions.
3. Derive net pay from salary configurations, attendance parameters, and configured deductions.

Sequence Diagram:

1. **Finance Admin → API Controller:** Requests payslip generation for employee specifying target month and year.
2. **API Controller → Payroll Service:** Verifies requesting user holds explicit finance management permissions.
3. **Payroll Service → Database (ORM):** Queries existing Payslip records to ensure no duplicate entry exists for target period.
4. **Database (ORM) → Payroll Service:** Confirms absence of existing payslip for the given month/year.
5. **Payroll Service:** Fetches employee base salary structure and attendance deductions to calculate net payable salary.
6. **Payroll Service → Database (ORM):** Creates and stores computed Payslip record.
7. **Payroll Service → API Controller:** Returns newly generated payslip entity payload.

8. **API Controller → Finance Admin:** Returns 201 Created HTTP response with generated payslip details.

7. Audit Logging & Security Component

Purpose: Records data mutations in append-only log tables and manages temporary "Break-Glass" support impersonation.

Inputs: Entity modification triggers (create/update/delete), Super Admin support access requests.

Outputs: Immutable AuditLog records, support authorization access tokens, ImpersonationLog entries.

Dependencies: Data Access Layer (Django Signals / Middleware).

Internal Logic:

1. Intercept entity write transactions to store actor ID, target entity, action type, and timestamp in AuditLog.
2. Restrict API write and deletion endpoints on audit log records to preserve append-only security guarantees.
3. Require explicit tenant CEO approval for support access requests, granting time-limited access logged in ImpersonationLog.

Sequence Diagram:

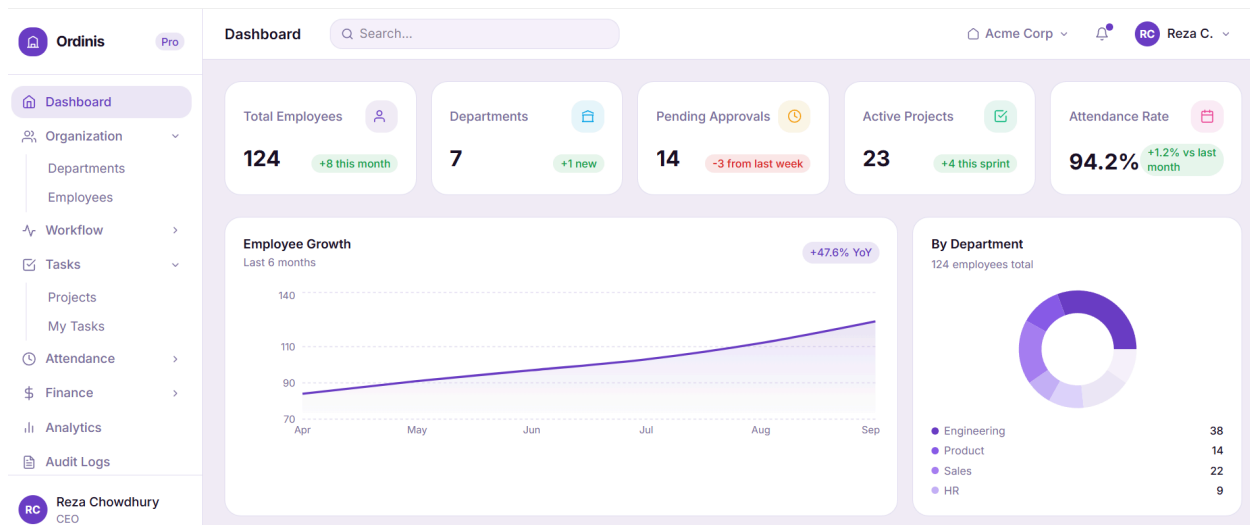
1. **Super Admin → API Controller:** Submits support access request specifying tenant_id and justification reason.
2. **API Controller → Security Service:** Forwards request to create an isolated impersonation authorization ticket.
3. **Security Service → Target Tenant CEO:** Sends notification alert requesting explicit "Break-Glass" access grant.
4. **Target Tenant CEO → Security Service:** Approves access request granting time-restricted support window.
5. **Security Service:** Generates temporary access token scoped strictly to the requested tenant environment and logs entry in ImpersonationLog.
6. **Security Service → API Controller:** Passes generated time-limited access session token.
7. **API Controller → Super Admin:** Returns 200 OK response with temporary support token and session expiry time.

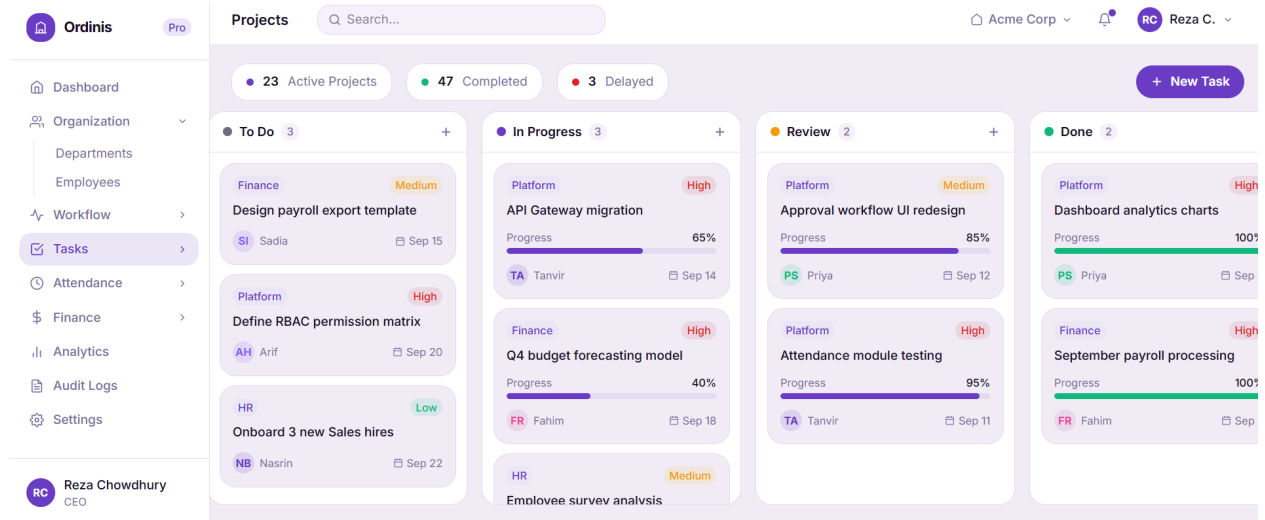
6. Interface Design

6.1 User Interface Design

The system's UI architecture follows a responsive multi-tenant SaaS dashboard design built for web (desktop-first) and mobile viewports. Key screens designed in Figma include:

- Authentication & Onboarding Screen: Login portal supporting single sign-on (SSO), domain-based tenant routing, and multi-factor authentication (MFA) prompts.
- CEO/Admin Executive Dashboard: High-level overview displaying real-time organization metrics, department performance graphs, pending approvals counter, and audit event logs.
- Manager Reporting Workspace: Dynamic team tree view, task allocation board (Kanban style), and approval queue for leave/expense claims.
- Employee Self-Service Portal: Personal attendance check-in/out widget, task progress update timeline, leave application interface, and payslip preview/download tab.
- Super Admin Support Console: Tenant management panel, "Break-Glass" impersonation request launcher, and system-wide security logs.





6.2 External Interfaces

Third-Party APIs

- **OAuth 2.0 / OpenID Connect APIs:** Google Workspace and Microsoft Azure AD for enterprise Single Sign-On (SSO) authentication.
- **Twilio API:** Provides SMS gateway capabilities for multi-factor authentication (MFA) and urgent critical event notifications.
- **SendGrid / AWS SES API:** Transactional email service for system invites, approval routing alerts, and monthly payslip deliveries.

External Systems

- **Cloud Object Storage (Amazon S3 / DigitalOcean Spaces):** Secure, encrypted bucket storage hosting uploaded identity documents, expense receipts, and generated PDF payslips.
- **Payment Gateways / Banking APIs:** External processing interfaces for direct salary disbursement integrations and employee expense reimbursements.

6.3 API Design

Endpoint	Method	Request Payload	Response Payload	Description

/api/v1/auth/login/	POST	{ "email": "...", "password": "..." }	{ "access": "...", "refresh": "...", "user": {...} }	Authenticate s user credentials and issues signed JWT tokens.
/api/v1/auth/refresh/	POST	{ "refresh": "..." }	{ "access": "..." }	Renews expired access token using a valid refresh token.
/api/v1/employees/	GET	Headers: Authorization Bearer	[{ "id": 1, "name": "...", "reports_to": 2, ... }]	Retrieves filtered employee tree list scoped to caller permissions.
/api/v1/approvals/	GET	Query Params: status=pending	[{ "id": 10, "type": "Leave", "step": 1, ... }]	Lists pending approval requests assigned to current user/delegate .

/api/v1/approvals/{id}/action/	POST	{ "action": "APPROVE", "comment": "... " }	{ "status": "Processed", "current_step": 2 }	Submits approval or rejection decision on an approval instance.
/api/v1/tasks/{id}/progress/	POST	{ "completion_pct": 75, "log": "... " }	{ "id": 5, "status": "In Progress", ... }	Logs daily task progress percentage and updates activity status.
/api/v1/attendance/check-in/	POST	{ "timestamp": "...", "location": "... " }	{ "id": 88, "check_in": "...", "status": "OK" }	Logs daily employee attendance check-in timestamp.
/api/v1/payroll/generate/	POST	{ "employee_id": 4, "month": 8, "year": 2026 }	{ "payslip_id": 501, "net_salary": 4500.00 }	Generates monthly payslip record (restricted to Finance role).

6.4 Hardware Interfaces

The software system operates purely over web and mobile software layers; no direct physical hardware interfaces (such as direct microcontrollers or embedded sensors) are integrated natively into the core application logic. Bio-metric devices (e.g., fingerprint or facial scanners) interface

indirectly through secure HTTP/REST external edge gateway proxies that post timestamp payloads directly to the attendance API endpoint /api/v1/attendance/check-in/.

6.5 Communication Interfaces

Protocols

- **HTTPS (HTTP over TLS 1.3):** Mandatory transport layer protocol enforcing end-to-end payload encryption for all API requests, web portal accesses, and administrative sessions.
- **WSS (WebSocket Secure):** Persistent bidirectional channel utilized for sending real-time notifications, approval alerts, and active session updates to front-end clients.
- **TCP/IP:** Foundational network protocol facilitating transport layer communication between application servers, database replicas, and caching nodes (Redis).

Message Formats

- **JSON (JavaScript Object Notation):** Primary lightweight data exchange format for all client-server RESTful API requests and responses. All JSON keys follow strict snake_case naming conventions.
- **JWT (JSON Web Tokens):** Formatted cryptographically signed string passed via HTTP Authorization: Bearer <token> headers to assert tenant context and user authentication identity.

7.1 Authentication & Authorization

Authentication Method:

JWT: Short-lived access tokens (15 mins) and long-lived refresh tokens (7 days) stored in HttpOnly, SameSite=Strict, Secure cookies.

MFA & SSO: TOTP-based MFA for sensitive operations; OAuth 2.0 / OIDC for Google Workspace and Azure AD SSO.

RBAC & Multi-Tenancy:

Tenant Isolation: Enforced at ORM query level using company_id.

Role-Based Access: Access granted via granular permission codenames assigned to user roles.

Hierarchy Scoping: Managers can view/edit data only for direct/indirect reports based on Employee.reports_to.

7.2 Data Security

Encryption Standard:

In Transit: HTTPS / TLS 1.3 enforced for all communications with HSTS enabled.

At Rest: AES-256 for database volumes and S3 storage; Fernet field-level encryption for sensitive PII.

Key Management:

Managed via AWS KMS or HashiCorp Vault.

Credentials kept out of source code and rotated automatically every 365 days.

7.3 Security Threat Analysis

Identified System Threat	STRIDE Category	Mitigation Strategy
Cross-Tenant Data Leakage (IDOR)	Information Disclosure	Enforce global ORM filtering (<code>company_id=request.user.company_id</code>) on all database queries.
Session Hijacking via Stolen Token	Spoofing	Use short-lived JWT access tokens (15 mins) and store refresh tokens in <code>HttpOnly</code> , <code>SameSite=Strict</code> cookies.
Audit Log or Payroll Record Tampering	Tampering	Apply append-only database rules and disable UPDATE/DELETE API endpoints for audit tables.
User Repudiation of Critical Actions	Repudiation	Record complete audit logs capturing actor ID, timestamp, IP address, and payload diffs for every state change.

Brute-Force & DoS Attacks	Denial of Service	Implement Redis-backed rate limiting (max 5 failed login attempts per minute) and WAF protection.
Privilege Escalation to Admin/Finance	Elevation of Privilege	Enforce RBAC middleware checking explicit permission codenames before API controller execution.
SQL Injection & XSS Attacks	Tampering	Use Django ORM parameterized queries and auto-escape dynamic HTML output in the React front-end.
Abuse of "Break-Glass" Support Access	Elevation of Privilege	Require explicit tenant CEO approval for temporary support access and record all actions in an ImpersonationLog.

8. Performance Design

8.1 Performance Requirements

Response Time: Standard API operations (such as attendance check-ins and task status updates) maintain an execution response time of ≤ 200 ms for 95% of requests. Complex analytical processes, including bulk monthly payroll calculations and aggregate organizational attendance reports, complete within ≤ 2 seconds.

Throughput: The backend architecture supports a peak throughput of 500 requests per second (RPS) during high-traffic operational windows (such as morning check-ins). The data layer handles up to 1,000 write transactions per minute without locking issues or request queue degradation.

Concurrent Users: The infrastructure is engineered to sustain 1,000 active concurrent user sessions without degradation in performance, supporting an initial target scale of up to 10,000 total registered employee accounts across all client tenants.

8.2 Scalability Strategy

Horizontal Scaling: Application logic is decoupled into stateless Docker containers managed via Kubernetes / AWS ECS auto-scaling groups. New application container nodes automatically spin up whenever cluster CPU usage exceeds 70% or memory utilization exceeds 80%. Database read workloads are offloaded to PostgreSQL Read Replicas.

Load Balancing: An NGINX / AWS Application Load Balancer (ALB) operates at the ingress layer to terminate HTTPS connections and evenly distribute incoming API traffic across active container instances using a Round-Robin routing algorithm backed by continuous health checks.

Caching Strategy: A Redis in-memory cache layer stores frequent authorization checks, active RBAC permission matrices, JWT token revocation lists, and tenant settings. This drastically minimizes direct database query hits for repetitive operations.

8.3 Capacity Planning

Database Storage: The system initializes with a 50 GB primary PostgreSQL NVMe SSD storage instance on AWS RDS, configured with automated storage auto-scaling to seamlessly expand up to 250 GB in Year 1 alongside daily automated backups.

Object Storage: Document storage (such as employee identity documents, expense receipts, and generated PDF payslips) starts with a 100 GB AWS S3 bucket. Automated lifecycle rules transition files older than 1 year to S3 Glacier for cost-effective long-term archiving up to 1 TB.

Compute Capacity: Application server capacity dynamically scales between a baseline of 2 container instances during normal operations up to 8 container instances during peak business load windows.

9. Error Handling & Logging

- **9.1 Error Handling Strategy:** Global Django REST Framework exception handlers intercept application errors and return standardized JSON responses ({ "success": false, "error": { ... } }) while hiding stack traces under generic 500 codes for security. Asynchronous background tasks (like sending emails or generating reports) use Celery and Redis to perform exponential backoff retries, while failed external API calls retry up to 3 times before routing to a Dead Letter Queue (DLQ).
- **9.2 Logging Strategy:** System activity is logged across five standard levels (DEBUG, INFO, WARNING, ERROR, CRITICAL), capturing structured JSON output (stdout/stderr) that feeds directly into a centralized ELK stack or Grafana Loki pipeline. Real-time application monitoring and crash reporting are handled by Sentry and

Prometheus/Grafana, triggering automated alerts to Telegram/Slack webhooks whenever system metrics or failure rates breach safety thresholds.

10. Deployment Design

The Ordinis system will be deployed as a web-based multi-tenant application using a React-based frontend and Django-based backend architecture. The system will follow a client-server deployment architecture where users access the React frontend through modern web browsers, while the Django backend provides RESTful APIs and handles authentication, authorization, tenant management, organizational hierarchy, approval workflows, task management, attendance, leave, payroll, notifications, audit logging, and other business operations.

The React frontend will be deployed as a production-built web application, while the Django backend will run as a server-side application using an application server such as Gunicorn or uWSGI. PostgreSQL will be used as the primary production database for storing company information, employee records, organizational structures, roles, permissions, tasks, approvals, attendance, leave, payroll, notifications, audit logs, subscription information, and other system data. SQLite will be used for local development and testing.

The major deployment components are:

- **Client Devices:** Desktop computers, laptops, tablets, and smartphones with modern web browsers.
- **Frontend Server:** Hosts the production React application and delivers the user interface to clients.
- **Backend Server:** Runs Django and Django REST Framework and provides REST APIs for communication between the frontend and database.
- **Database Server:** PostgreSQL stores persistent organizational, employee, workflow, attendance, leave, payroll, and other business data.
- **Authentication Layer:** JWT-based authentication manages secure user access to protected system resources.
- **Service Layer:** Handles business logic such as approval processing, leave validation, task transitions, payroll processing, and other application operations.
- **Audit Logging:** Records important user and system activities for security, accountability, and monitoring.
- **Notification Component:** Manages system notifications and actionable activities for users.

The communication between the client and backend will use HTTPS in the production environment. RESTful APIs will use HTTP methods such as GET, POST, PUT/PATCH, and DELETE. JSON will be used as the primary data-exchange format between the React frontend and Django backend.

Deployment Architecture

The deployment architecture consists of three major application layers:

1. Presentation Layer

- React/Vite
- HTML5
- CSS and Tailwind CSS
- JavaScript
- Browser-based user interface

2. Application Layer

- Django
- Django REST Framework
- REST API
- Authentication and authorization
- Tenant management
- Organization and employee hierarchy
- RBAC and permission management
- Approval workflow management
- Task and project management
- Attendance and leave management
- Payroll and expense management
- Notification and Action Inbox
- Audit logging and analytics
- Subscription and billing management

3. Data Layer

- PostgreSQL for production
- SQLite for development and testing
- Persistent storage for company, employee, organization, role, permission, task, approval, attendance, leave, payroll, notification, audit, and subscription data.

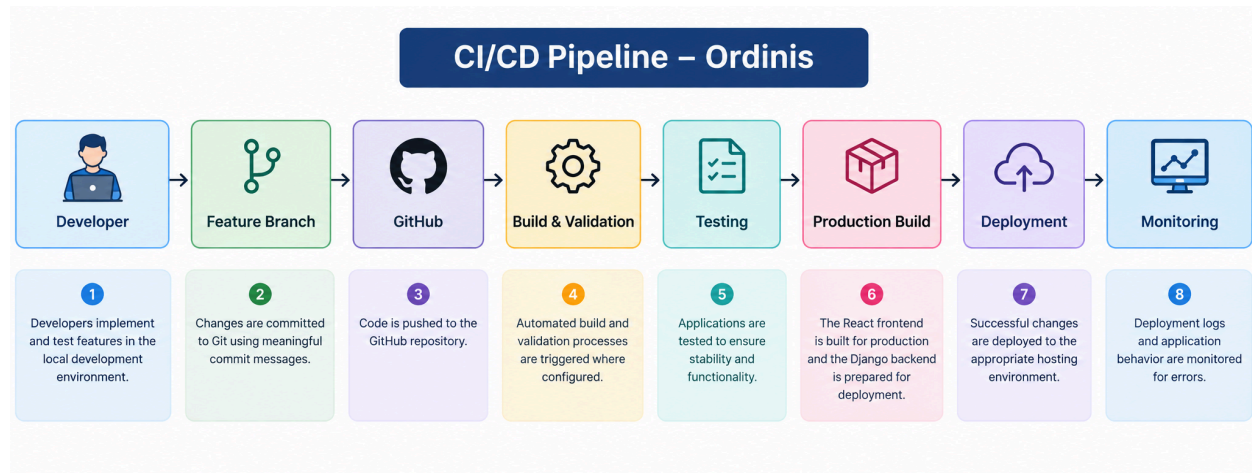
Deployment Node Mapping

Deployment Node	Technologies	Major Components
Client Device	Web Browser, React	Employee, Manager, Administrator, and other user interfaces
Frontend Hosting Server	React/Vite, HTML, CSS, JavaScript, Tailwind CSS	User interface and client-side application
Backend Server	Django, Django REST Framework	REST API, authentication, authorization, business logic, and application services
Database Server	PostgreSQL	Company, employee, organization, task, approval, attendance, leave, payroll, and other persistent data
Authentication Layer	JWT, Django Authentication	User authentication and protected access
Service Layer	Django Service Modules	Business rules, approval processing, leave, payroll, task, and related operations
Audit & Logging Layer	Django Logging, Audit Log	Security and business activity records
Notification Layer	Django Notification Services	User notifications and Action Inbox activities

10.1 Deployment Strategy

CI/CD Pipeline

The project will use a Git-based development workflow for source-code management and deployment. The project repository will contain the React frontend and Django backend in separate or clearly organized project directories. The proposed CI/CD workflow is:



The CI/CD process will perform the following activities:

1. Developers implement and test features in the local development environment.
2. Changes are committed to Git using meaningful commit messages.
3. Code is pushed to the GitHub repository.
4. Automated build and validation processes are triggered where configured.
5. The React frontend is built for production.
6. The Django backend and its dependencies are prepared for deployment.
7. Database migrations are checked and applied carefully when required.
8. Successful changes are deployed to the appropriate hosting environment.
9. Deployment logs and application behavior are monitored for errors.
10. Failed builds or validation checks prevent the affected version from being promoted.

Versioning Strategy

Git and GitHub will be used for source-code version control and team collaboration.

The following branching strategy will be used:

- **main**: Stable and production-ready version of the Ordinis system.

- **develop:** Integration branch where completed features are combined and tested before release.
- **feature branches:** Used for developing individual features, modules, or bug fixes.
- **Release versions:** Stable releases may be identified using version numbers such as V1.0, V1.1, and V2.0 to maintain a clear history of system changes.

10.2 Environment Configuration

Ordinis uses separate configurations for development and production environments.

Configuration	Development	Production
Frontend	React/Vite development server	Built React application
Backend	Django development server	Gunicorn/uWSGI
Database	SQLite	PostgreSQL
Operating System	Developer machine	Linux server
Debug Mode	Enabled for development	Disabled
API	Local REST API	HTTPS REST API
Authentication	JWT	Short-lived JWT with refresh rotation
Logging	Development logs	Structured application and audit logs
Secrets	Local environment configuration	Secure environment variables/secrets

Sensitive configuration values such as database credentials, secret keys, and authentication settings should not be hardcoded in the source code. They should be provided through environment-specific configuration.

10.3 Rollback Strategy

The rollback strategy is designed to restore the previous stable version if a deployment causes critical problems.

The main rollback steps are:

1. Identify the deployment problem using application and error logs.
2. Stop or isolate the faulty release if necessary.
3. Restore the previously stable application version.
4. Verify frontend, backend, authentication, and database connectivity.
5. Check important operations such as employee management, leave, attendance, and payroll.
6. Restore the database from a verified backup if a database migration has caused an unrecoverable problem.
7. Confirm that the system is working correctly before continuing normal operation.

Database changes should be designed carefully so that application rollback does not unnecessarily cause data loss.

11. Design Constraints

Regulatory Requirements

Since Ordinis handles employee personal information, organizational data, attendance, leave, payroll, and other sensitive business records, the system must protect this information from unauthorized access and modification. The system must:

- Protect Personally Identifiable Information (PII), employee records, salary information, attendance records, leave records, and other sensitive company data.
- Restrict sensitive employee and payroll information to authorized users according to their assigned roles and permissions.
- Maintain strict tenant isolation so that users of one company cannot access or modify data belonging to another company.
- Prevent unauthorized modification of important organizational, attendance, leave, and payroll records through server-side authorization and validation.
- The permission system must be enforced at the backend level; hiding buttons or features in the frontend alone must not be considered sufficient security.
- Audit logs must be maintained as append-only records so that important system activities can be tracked and reviewed.
- Break-Glass support access must require appropriate approval, be time-limited, and be fully recorded in the audit trail.
- Important business operations such as leave, attendance, and payroll updates must maintain transactional consistency.

Technology Constraints

The Ordinis system will be developed using a modern web-based technology stack consisting of:

- **React/Vite** – Frontend user interface.
- **Tailwind CSS** – Responsive and consistent user interface styling.
- **Django** – Backend application framework.
- **Django REST Framework** – REST API development.
- **Django ORM** – Database management and communication.
- **SQLite** – Development and testing database.
- **PostgreSQL** – Production relational database.
- **JWT** – Token-based authentication.

Also,

- The system should use open-source technologies and libraries wherever practical to keep development and deployment costs low.
- Frontend and backend communication must use REST APIs with JSON.
- Production communication must use HTTPS to protect data during transmission.
- Business logic should be maintained in a dedicated service layer to keep the system modular and maintainable.
- Every tenant-specific database operation must respect the `company_id` to maintain data isolation.
- The organizational hierarchy, roles, permissions, and approval chains should remain configurable rather than being permanently hardcoded.

Budget Constraints

Ordinis is developed as an academic project without external funding. Therefore, the system is designed to maintain a low-cost technology footprint. All core technologies are open-source or can be used through free or low-cost development environments:

- **Database:** SQLite can be used during development, while PostgreSQL can be used for production deployment using suitable low-cost or free-tier infrastructure.
- **Frameworks & Libraries:** React, Tailwind CSS, Django, Django REST Framework, and supporting open-source libraries do not require commercial licensing fees.
- **Hosting:** Free-tier or low-cost hosting can be used for development and academic demonstration.
- **Version Control & Collaboration:** Git and GitHub can be used for source-code management and team collaboration.
- **Future Costs:** Production-scale hosting, database storage, monitoring, backup services, and other external services may create recurring costs if Ordinis is expanded beyond academic use.

Timeline Constraints

The Ordinis project is developed within a fixed academic delivery period. The limited development time requires the team to prioritize the core features of the system, including organization management, employee hierarchy, RBAC, approval workflows, tasks, attendance, leave, payroll, notifications, audit logging, and subscription management.

The fixed timeline limits the implementation of advanced integrations, extensive production optimization, dedicated mobile applications, and other future features. Therefore, the current release focuses on providing a functional, secure, and maintainable system prototype while keeping the architecture flexible for future development.

12. Assumptions & Dependencies

Assumptions

The design and deployment of Ordinis are based on the following assumptions:

- Companies using Ordinis are assumed to operate as separate tenants, with each company's data isolated from other companies.
- Users are assumed to have valid accounts and appropriate permissions before accessing protected system features.
- Company administrators are assumed to provide accurate information when creating departments, designations, employees, roles, permissions, and approval chains.
- Employees and administrators are assumed to provide accurate attendance, leave, task, and payroll-related information, as this data is used for system processing and reporting.
- Users are assumed to have access to a modern web browser and sufficient internet connectivity to use the web-based platform.
- The **reports_to** relationship is assumed to correctly represent the reporting structure between employees.
- Approval chains are assumed to be configured according to the actual organizational workflow of each company.
- Companies using the payroll module are assumed to maintain accurate salary and employee payment information.
- The production environment is assumed to have appropriate database backup, recovery, and security configurations.
- The current release is designed primarily as a web-based platform and does not assume the availability of a dedicated mobile application.

External System Dependencies

The Ordinis system depends on the following frameworks, tools, and services:

- React/Vite: Used to develop the frontend interface and interactive user experience.
- Tailwind CSS: Used for responsive and consistent user interface styling.
- Django & Django REST Framework: Used for backend development, business logic, and REST API implementation.
- Django ORM: Used to communicate with the relational database and manage database operations.
- PostgreSQL: Used as the target production database for storing organizational and business data.
- SQLite: Used as the development and testing database.
- JWT: Used for token-based user authentication and session management.
- Git & GitHub: Used for version control, collaborative development, and maintaining the project codebase.
- Linux Server: Used as the target production environment.
- Gunicorn/uWSGI: Used to serve the Django application in the production environment.

Third-Party Libraries

The system may depend on the following supporting libraries and technologies:

- React: Used to build a component-based frontend application.
- React Router: Used for client-side navigation between application pages.
- Tailwind CSS: Used for responsive interface design and styling.
- Django REST Framework: Used for developing RESTful backend APIs.
- JWT Library: Used for token-based authentication.
- PostgreSQL Driver: Used for communication between Django and the PostgreSQL database.
- Axios or Fetch API: Used for frontend-backend API communication, depending on the final implementation.
- Gunicorn/uWSGI: Used for production application serving.

13. Traceability Matrix

13.1 Functional Requirements Traceability Matrix

Requirement ID	Design Component	Remarks
FR 1.1	Authentication Module, React Login Interface, Django Authentication API, User Model	Provides secure authentication for system users.
FR 1.2	Authentication Module, JWT, Session Management	Supports secure login, logout, and token management.
FR 1.3	Authorization Module, Role Model, Permission Model, Protected API Endpoints	Restricts system access according to assigned roles and permissions.
FR 2.1	Tenant Management Module, Company Model, React Company Interface	Allows companies to be created and managed as separate tenants.
FR 2.2	Organization Module, Department Model, Department Interface	Supports department creation and management.
FR 2.3	Organization Module, Designation Model, Designation Interface	Supports creation and management of employee designations.
FR 2.4	Employee Module, Employee Model, Employee Interface	Allows administrators to create and manage employee information.

FR 2.5	Employee Hierarchy Module, Employee Model, reports_to Relationship	Maintains the reporting relationship between employees.
FR 3.1	RBAC Module, Role Model, Permission Model	Supports role and permission management.
FR 3.2	Authorization Service, Permission Middleware, Protected Routes	Restricts access to authorized system operations.
FR 3.3	Hierarchy-based Authorization Module, Employee Hierarchy	Controls employee visibility according to organizational hierarchy.
FR 4.1	Approval Chain Module, Approval Chain and Approval Step Models	Supports configurable approval chains.
FR 4.2	Approval Workflow Module, Approval Instance and Approval Action Models	Tracks approval requests and actions.
FR 4.3	Delegation Module, Delegation Rule Model	Supports controlled delegation of approval responsibilities.
FR 5.1	Project and Task Module, Project Model, Task Model	Supports project and task management.
FR 5.2	Task Management Interface, Task API, Task Model	Allows users to create, update, assign, and manage tasks.

FR 5.3	Task Progress Module, Task Progress Log	Records and tracks daily task progress.
FR 6.1	Attendance Module, Attendance Model, Attendance API	Records and manages employee attendance.
FR 6.2	Leave Management Module, Leave Type and Leave Request Models	Supports employee leave requests and leave types.
FR 6.3	Leave Approval Module, Approval Workflow, Leave Service	Processes and manages leave approval.
FR 6.4	Leave and Attendance Service, Transaction Management	Maintains consistency between leave, attendance, and related operations.
FR 7.1	Payroll Module, Salary Structure Model, Payroll Service	Supports employee salary structure management.
FR 7.2	Payslip Module, Payslip Model, Payroll Service	Generates and manages employee payslips.
FR 7.3	Expense and Reimbursement Module, Expense and Reimbursement Models	Supports employee expense and reimbursement management.
FR 8.1	Notification Module, Notification Model, Notification API	Provides notifications for important system activities.

FR 8.2	Action Inbox, Approval and Notification Components	Displays actionable pending activities to users.
FR 9.1	Audit Module, Audit Log Model, Audit Service	Records important user and system activities.
FR 9.2	Analytics Module, Company Health Snapshot	Provides basic organizational and operational insights.
FR 10.1	Subscription Module, Subscription Model, Invoice Model	Supports company subscription and billing management.
FR 10.2	Platform Administration Module, Support Access Request	Supports controlled platform-level administrative access.
FR 10.3	Break-Glass Access Module, Impersonation Log	Provides time-limited and fully logged support access.

13.2 Non-Functional Requirements Traceability Matrix

Requirement ID	Design Component	Remarks
NFR 1.1	React UI, Django REST API, PostgreSQL, Pagination	Supports acceptable response time during normal system operation.
NFR 1.2	Database Indexing, Efficient Queries, Service Layer	Improves system performance for frequently used operations.

NFR 2.1	JWT Authentication, Password Hashing, Authentication Module	Protects user accounts and authentication information.
NFR 2.2	RBAC, Permission Middleware, Protected API Endpoints	Restricts sensitive system functions according to permissions.
NFR 2.3	Tenant Manager, <code>company_id</code> , Tenant-aware Database Queries	Prevents unauthorized cross-tenant data access.
NFR 2.4	HTTPS, Secure Configuration, JWT	Protects data during communication and authentication.
NFR 3.1	React UI Components, Consistent Layout, Tailwind CSS	Provides a simple and consistent user interface.
NFR 3.2	React Navigation, Dashboard, Actionable Error Messages	Provides clear navigation and understandable system feedback.
NFR 4.1	Django ORM, PostgreSQL, Foreign Keys, Validation	Maintains accurate and consistent business data.
NFR 4.2	ACID Transactions, Service Layer, Database Constraints	Protects data consistency during important business operations.
NFR 5.1	Layered Architecture, Service Layer, Modular Components	Supports maintainability and separation of responsibilities.

NFR 5.2	Configurable RBAC, Approval Workflow, Organizational Hierarchy	Allows business rules to be changed without major architectural changes.
NFR 6.1	PostgreSQL, Pagination, Layered Architecture	Supports future growth in users and organizational data.
NFR 6.2	Stateless REST API, Environment Configuration	Supports portability between development and production environments.
NFR 7.1	Database Backup, Error Handling, Transaction Management	Supports reliable operation and recovery from system failures.
NFR 7.2	Audit Log, Append-only Logging	Maintains accountability and historical activity records.
NFR 8.1	Linux Server, PostgreSQL, Gunicorn/uWSGI	Supports stable production deployment.
NFR 8.2	Environment-based Configuration	Keeps development and production configurations separate.
NFR 9.1	Break-Glass Access, Support Access Request, Impersonation Log	Ensures sensitive support activities are controlled and auditable.

NFR 9.2	Modular React and Django Architecture	Allows future modules and features to be added with minimal impact on existing components.
---------	---------------------------------------	--

14. Appendices

Appendix A – Glossary

ALB (Application Load Balancer): A Layer 7 load balancer service on AWS that distributes incoming HTTP/HTTPS traffic across backend target instances based on request attributes.

API (Application Programming Interface): A set of defined rules and protocols that allow client frontends and backend services to communicate over HTTPS.

Break-Glass Access: An emergency access mechanism allowing temporary elevated support access to a tenant's environment, subject to explicit CEO authorization and mandatory audit logging.

Celery: An asynchronous task queue/job queue based on distributed message passing, used for executing background processes like email processing and payroll generation.

EKS (Elastic Kubernetes Service): A managed Kubernetes container orchestration service provided by AWS for scaling and deploying containerized applications.

IDOR (Insecure Direct Object Reference): A security vulnerability where an application provides direct access to objects based on user-supplied input, mitigated via tenant-scoped database filters.

JWT (JSON Web Token): An open, industry-standard (RFC 7519) compact and self-contained method for securely transmitting information between parties as a JSON object.

Multi-Tenancy: A software architecture where a single software instance serves multiple distinct customer organizations (tenants) while strictly isolating their data.

ORM (Object-Relational Mapping): A programming technique (Django ORM) that maps Python object models directly to relational database tables.

RBAC (Role-Based Access Control): An approach to restricting system access to authorized users based on granular permission codenames assigned to user roles.

RDS (Relational Database Service): A managed relational database web service provided by AWS hosting the system's PostgreSQL instance.

SDS (Software Design Specification): A comprehensive architectural document detailing the complete technical design, data schema, interfaces, and security model of a software system.

STRIDE: A threat modeling framework developed by Microsoft used to categorize security threats into Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privilege.

TOTP (Time-based One-Time Password): A temporary passcode generated by an algorithm using the current time of day as a factor, used for multi-factor authentication (MFA).

Appendix B – Diagrams

This section compiles all primary system diagrams referenced throughout the design document for architectural overview and lab submission verification:

1. System Architecture Diagram (Section 3): High-level overview illustrating client-side React SPA communicating with backend Django API nodes, Celery workers, and data storage systems (RDS PostgreSQL, Redis, AWS S3).
2. Entity-Relationship Diagram (ERD) (Section 4): Database schema mapping key entities including **Company**, **Employee**, **Role**, **Permission**, **ApprovalInstance**, **Task**, **Attendance**, **Payslip**, and **AuditLog**.
3. Component Interaction Sequence Diagrams (Section 5.2): Sequence workflows detailing message exchanges for User Authentication, RBAC Permission Enforcement, Multi-step Approval Routing, and Break-Glass Support Access.
4. UI Wireframe Layout (Section 6.1): Layout architecture of the Executive Dashboard and Manager Workspace interfaces.
5. Deployment Architecture Diagram (Section 10.1): Cloud topology illustrating client edge networks, ALB routing, isolated private subnets, EKS container nodes, and database clusters.

Appendix C – Supporting Documents

Software Requirements Specification (SRS): The foundational requirements document specifying functional requirements (FR1 through FR25) and non-functional targets for the Ordinis Organization Operating System.

Database Schema Migration Scripts: DDL scripts and Django ORM migration files used to initialize multi-tenant database tables, indexing strategies, and foreign key constraints.

OpenAPI / Swagger 3.0 Specification: Complete interactive API documentation file (openapi.yaml) detailing endpoints, request payloads, response schemas, and error definitions.

Figma UI/UX Prototype Link: Interactive high-fidelity wireframe prototype demonstrating navigation flows across Executive, Manager, and Employee dashboards.

Academic Lab Rubric Compliance Matrix: Cross-reference checklist mapping each section of this SDS against the CS/SE lab evaluation criteria for final submission.